

The development of digital amplifier with active cross-over and room equalisation

Abstract

My project concerns with using active digital correction and cross-over filters in conjunction with class-D amplification to try to reproduce sound as accurately as possible in a room environment. Digital audio signal can be filtered in digital domain, before any amplification takes place. This can be done with high accuracy and allows for much greater flexibility than passive components. Separated high and low frequency signals can then be amplified using any method, including class D topology, and fed directly to the speakers. But obtaining perfect sound coming out of the loudspeakers is not enough. The room creates a lot of acoustic reflections, which can significantly alter the frequency spectrum of the sound reaching the user. This can be favourable in some cases, however I'm concerned with reproducing sound as close to the original as possible.

A simulation model of room acoustics has been designed and evaluated in Matlab. The output of the simulation, as well as recorded impulse responses, have been used to develop equalisation digital filter. Such filter would pre-equalise the digital signal before it is played through the speakers, to obtain the sound as close to the original in a specified listening position.

Contents

Abstract.....	1
Introduction	2
Acoustic ray tracing algorithm	3
Performance considerations.....	6
Limitations of the model.....	6
Proposed improvements.....	6
Simulation results	6
Impulse response cancellation.....	7
Maths	9
Further work	13
Conclusion.....	13
Revised schedule.....	13
Bibliography	13

Introduction

The original simplified diagram of the system is shown in Figure 1.

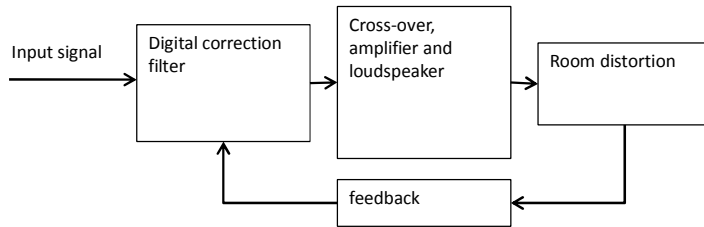


Figure 1

First of all, I decided to concentrate on modelling the room impulse response.

I have considered a number of ways to simulate sound propagation in a room. The most important feature of such simulation is to obtain an acoustic impulse response in a particular location given the positions and type of sound sources and dimensions of the room.

There are few approaches to doing this. I have chosen acoustic ray tracing approach. I will discuss its advantages and shortcoming, as well as present a brief overview of other approaches.

I chose this approach as it seems to be the most flexible and powerful one.

Other two methods are image source model and acoustical Boundary Element Method (BEM).

Image source model is only valid for rectangular or very simple enclosures.

BEM is a very powerful but extremely complex method, however it does not directly provide impulse response of the room as the output [1].

Why simulate room acoustics and not just measure it? Provided that simulation model is quite accurate, it gives a lot more flexibility than a real room. By interactively changing simulation parameters such as room geometry and materials, we can see their effect on sound within the room. By making a lot of controlled changes and observing the effect, we can gain good insight into how acoustics is defined by the room. Making the changes to the actual room would require building works which is time-consuming and costly.

Another demand for acoustics simulation is modelling of environments that do not yet exist, such as music halls and listening rooms. Before expensive building works begin, we can get insight to what the newly build environment will sound like, and make any corrections.

Acoustic ray tracing algorithm

The generic acoustic ray tracing algorithm is described below

1. Define room boundary surfaces and surface materials
2. Start with an impulse $\delta(t)$ or $\delta[n]=[1, 0, 0 \dots]$ for discrete system
3. Create a sound ray with origin R_0 and direction vector R_d . For each ray store parameters:
 - a. Current travel distance
 - b. Spectrum or time domain representation
4. Use frequency-dependant directivity of the source to modify the initial amplitude and spectrum of the ray.
5. Trace Nth ray to the intersection with a surface
6. Calculate the absorption and phase change caused by the surface
7. Keep bouncing Nth ray until the total power in the ray falls to small threshold value.
8. Check if ray intersects with receiver. If it does, add time-delayed impulse to the output impulse time vector
9. Repeat for a large number of angularly spaced out rays

The algorithm I implemented in Matlab is described in more detail below. Figure 2 shows the Matlab structure I used to store information defining a room.

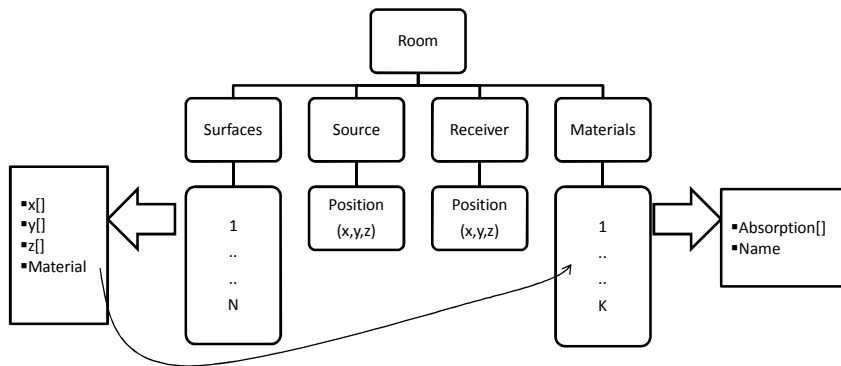


Figure 2 – structure for storing information about a room

Every element in array surfaces has a list of coordinates of vertices of a surface, and an integer number that defines what material surface is made of. Every material has array with absorption values ranging from 0 to 1 for the frequencies 125, 250, 500, 1000, 2000 and 4000 Hz. The values are taken from [2].

Condition that is implied but is not checked is that surfaces must form a closed volume. Otherwise some rays might escape into infinity. Holes in the surfaces such as open windows can be defined as surfaces with absorption of 1 at all frequencies.

Another important structure of the algorithm is a **ray**. This structure has fields R_0 , which is origin vector, and R_d , normalised direction vector of the ray.

Algorithm can be divided into geometrical part and acoustical part. Initially, when tens of thousands of rays are fired to see which ones intersect with the receiver, it is computationally expensive to calculate acoustical variables for every reflection. Therefore, the first pass determines which rays intersect with the receiver eventually and stored their R_d . This is sufficient to recreate the ray for the acoustic pass of the algorithm.

Majority of audio recordings are made in 16bit/44.1KHz format. To make the output of the model compatible, I decided to use fundamental sampling frequency of the system equal to 44.1KHz. This corresponds to maximum signal frequency of 22.05KHz.

For the frequency domain I decided to use 882 frequency steps from 0 Hz to 22050 Hz in 25Hz increments. 882 is not a power of 2, so isn't ideally suited for FFT calculation, however I found that there's no speed penalty as Matlab internally optimises the calculations.

I have considered sampling frequency in semi-octave increments ($1/12^{\text{th}}$ of an octave for example), but this would have caused problems when converting to and from time domain.

Vector calculations for intersections of lines, spheres and planes are taken from [3].

Geometric pass:

1. Firstly, all surfaces are checked for being planar. This is necessary as in further calculations every surface is assumed to lie in one plane:

```
vcheck=cross(v1,v2); % compute cross-product of any 2 vectors
for i=4:sX           % for every remaining point in the plane
v=[X(i)-X(1),Y(i)-Y(1),Z(i)-Z(1)] %calculate the vector
if (dot(vcheck,v)~=0) %if dot product isn't 0 it means 2 vectors aren't exactly
perpendicular
Is_Planar=false; %which means that not all points are in the same plane
```

2. Room surfaces, source and receiver are displayed using Matlab patch commands, axis and view angle are set:

```
for i=1:num
    patch(room-surfaces(i).x,room-surfaces(i).y,room-surfaces(i).z,'w');
end
...
alpha(0.5);
view([-160 25]);
```

3. The array of propagation vectors Rd is created by the function ray_angle_array. Rays are angularly spaced based on horizontal and vertical start and stop and angles, and number of steps:

```
for ver=1:ver_steps
    ver_angle=vertical_start_angle+ver*vertical_angle_step;
    hor_projection=sin(ver_angle);
    dz=cos(ver_angle);
    for hor=1:hor_steps
        hor_angle=horizontal_start_angle+hor*horizontal_angle_step;
        dx=hor_projection*sin(hor_angle);
        dy=hor_projection*cos(hor_angle);
        Rd=[dx,dy,dz];
    ...
end
end
```

4. For each the ray the following operations are performed:

- I. For every surface, check if ray intersects with plane defined by the surface. Store the distance from origin to intersection.

```
vd=dot([plane.A plane.B plane.C],line.Rd);
v0=-(dot([plane.A plane.B plane.C],line.R0)+plane.D);
t=v0/vd;
if t<0 ... % no intersection
intersection_point(1)=line.R0(1)+t*line.Rd(1); % x
intersection_point(2)=line.R0(2)+t*line.Rd(2); % y
intersection_point(3)=line.R0(3)+t*line.Rd(3); % z
```

- II. Once list is complete, sort the list of intersection distances in ascending order.
- III. Starting from the plane with smallest distance, check if ray intersects the plane *inside* the polygon defined by the surface vertices. This is done by projecting the polygon and the intersection point coordinates onto a 2D plane in coordinates (u,v) [3]. A line is created starting from intersection in random direction, and intersections with polygon edges are counted. If intersection number is odd, the

point is inside the polygon, if its even, the point is outside. This is based on Jordan curve theorem. Once valid intersection surface is detected, go to next step, as there can be only one intersecting surface.

```
% convert plane from coordinates of 3 points to implicit form:
plane=Plane_3P_to_scalar(surface.x(1:3),surface.y(1:3),surface.z(1:3));
% Need to compare magnitudes of A B and C
if abs(plane.A)==max([abs(plane.A),abs(plane.B),abs(plane.C)]) % throw away x
    u=surface.y; % array of Y coords
    v=surface.z; % array of Z coords
    pu=point(2);
    pv=point(3);
    c_pu=check_point(2);
    c_pv=check_point(3);
elseif ... % do the same for plane.B and plane.C
[intersect_u intersect_v]=Intersection_line_segment(line,segment);
... Count intersections with edges of the polygon...
if (mod(intersections,2)==1)
    inside_polygon=true;
```

IV. Ray is geometrically reflected from the surface:

```
function [ R_inverted] = Reflect_ray(Rd,normal)
...
R_inverted=Rd-2*(dot(Rd,normal)*normal);
```

5. Steps I-IV are repeated a number of times, say 20. The number is chosen so that a ray energy will almost certainly decay to negligible value by that number of reflections. At each reflection a check is performed to see if the ray intersects with the receiver sphere:

```
%R0 and Rd origin and direction vector of a line
%S0 is the centre of the sphere
%Sr is the radius of the sphere
%1. B=2*(Xd*(X0-Xc)+Yd*(Y0-Yc)+Zd*(Z0-Zc))
B=2*(Rd(1)*(R0(1)-S0(1))+Rd(2)*(R0(2)-S0(2))+Rd(3)*(R0(3)-S0(3)));
%2. C=(X0-Xc)^2+(Y0-Yc)^2+(Z0-Zc)^2-Sr^2;
C=(R0(1)-S0(1))^2+(R0(2)-S0(2))^2+(R0(3)-S0(3))^2-Sr^2;
%3. Calculate discriminant
if (B^2-4*C)>0
    does_intersect=true;
```

Acoustic pass:

For every ray that was found to intersect with receiver in geometrical pass:

1. Initial spectrum is assigned to be flat, i.e. $H(f)=1$ for $f=0..20\text{KHz}$

```
new_ray.spectrum=ones(1,6);
```

2. Intersection and reflection operations are the same as for the first pass
3. At every reflection, modify ray spectrum vector by wall absorption coefficients. $H_{i+1} = H_i \times (1 - \text{absorption})$. Absorption vector is specified for 6 frequencies in octave increments.

```
new_ray.spectrum=new_ray.spectrum.*(1-room.materials(material_n).absorption) ;
```

4. Repeat until ray intersects with receiver, or until total power of the ray is below a threshold value, after which ray can be disregarded. $\sum_{n=1}^6 |H(n)|^2 < P_{th}$
5. Store the path distance d .
6. Now we need to extrapolate frequency spectrum from 6 points to full 882 points resolution. I have used Matlab function `interp1` with cubic interpolation method. I have set frequency amplitudes at 0 Hz and 22050 Hz (F_{max}) to 0, to represent the fact that signal should have no DC component and have roll-off at high frequencies.

```

absorption_spectrum_points=[0 125 250 500 1000 2000 4000 8000 16000 max(f)];
spectrum=[0 spectrum];
spectrum(10)=0;
full_spectrum=interp1(absorption_spectrum_points,spectrum,f,'cubic');

```

7. Convert the frequency representation of the signal to time domain using ifft.

```

% Mirror the array out_spectrum to obtain FFT-even function for real ifft output
time=ifft([0 wrev(out_spectrum) out_spectrum]);

```

8. Add the time domain impulse delayed by $\frac{d}{c}$ to the final impulse response.

Performance considerations

I used the *profiler* tool to investigate which functions take up the most execution time.

I found that drawing the rays takes up significant amount of time. Therefore once I ensured that rays are traced correctly, I disabled the ray line display.

I was surprised to see that functions `dot` and `cross` took up a lot of execution time. But examination of their code revealed that a lot of checking is performed to make these functions operate on any size of input data. I rewrote them just for the vectors of size 3, which sped up their execution by around 10 times.

Limitations of the model

Currently the model does not account for the following:

- Diffraction around hard edges
- Complex wall impedance
- Diffuse reflections

Another significant limitation is the proportion of the emitted rays that actually hit the receiver, which is approximately 1% for a simple rectangular room. For 360 rays vertically and horizontally (1° spacing, 129000 rays in total), only 1500 reached the receiver. The shape of the impulse response obtained with this number of rays is fairly reasonable. However this feature of the model predicts that it is possible to create a source that emits from 99% of its surface and no sound reaches the receiver, which is of course impossible. This inaccuracy can be overcome by including diffuse reflections and diffraction in the model.

Proposed improvements

Any accurate ray-tracing acoustical simulation must account for diffraction. Research [4] shows that results of simulation agree very closely with measured response if diffraction is accounted for, at least for simple environments. Geometric treatment of diffraction can be found in [5].

Instead of ray tracing, beam tracing could be used, where sound energy is traced as cones or pyramids instead of lines. That would make sure no energy is just “lost” in the room.

Simulation results

Figure 3 shows the simulated impulse response of a room, obtained with the algorithm described before. The parameters of the room were:

- 5.1m long by 3.2m wide and 2.07m tall
- Source: point source, located at (1,1,1)
- Receiver: sphere with radius 5cm located at (4, 2.3, 1.2)
- Rays: 360 in vertical and horizontal planes, total number $360^2 = 129600$

The overall shape and length of the simulated impulse are reasonable. However comparison with a real recorded impulse response, shown in Figure 4, reveals quite large differences in signal shape.

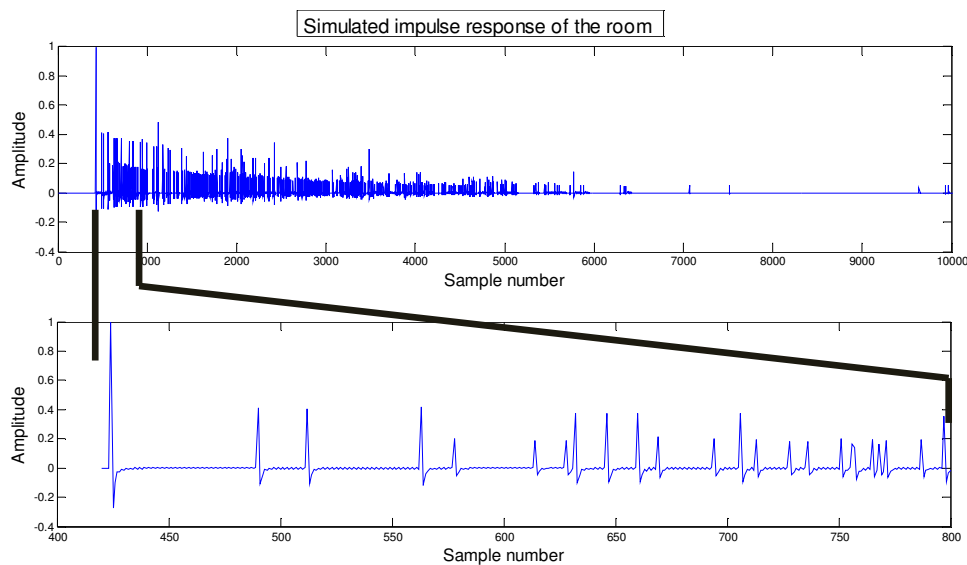


Figure 3 – simulated impulse response and a close-up view

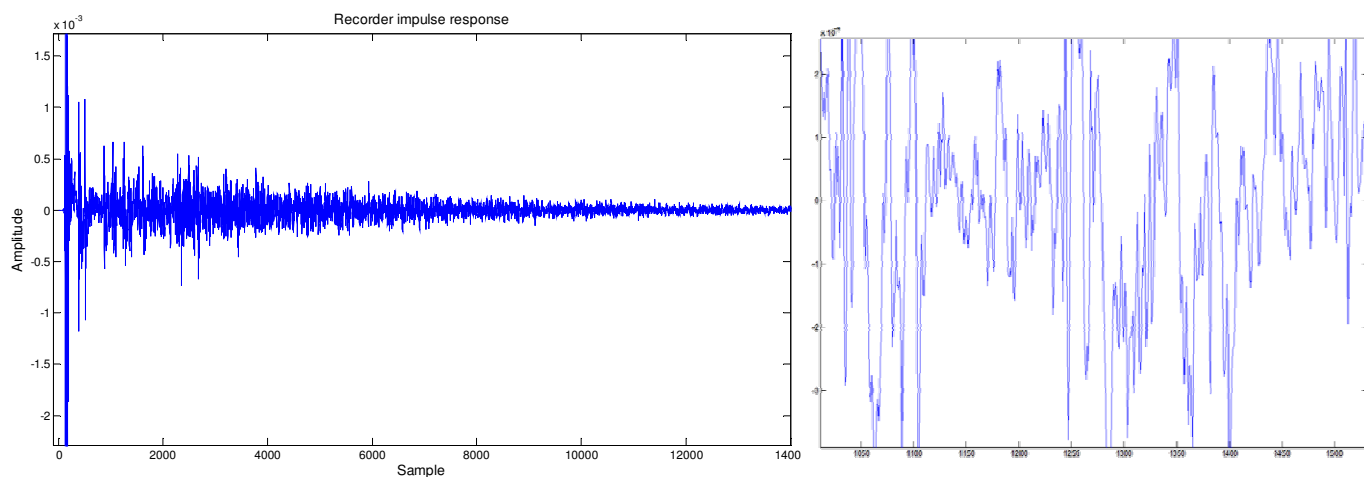


Figure 4 – real-life impulse response recorded in a stairway and a close-up view [6]

Impulse response cancellation

It is generally accepted that early reflections of the room are responsible for localisation of the sound source and tonal perception of the room. However late reflections are undesirable and can decrease speech intelligibility [2].

I assume that the sound stored on a digital media such a CD disk is recorded in non-anechoic environment, i.e. contains the acoustic response of the studio, and any deliberate effects musicians may have added to it. Therefore I consider it undesirable to add the listening room response onto it. So one way to hear the recording closest to the original would be to use headphones. Wearing high-quality headphones is obtrusive, so ideally I want to find a way to make loudspeakers sound as close the recording as possible at a given position.

Therefore I am aiming to shorten the impulse response of the room by pre-equalising the audio signal before the loudspeaker.

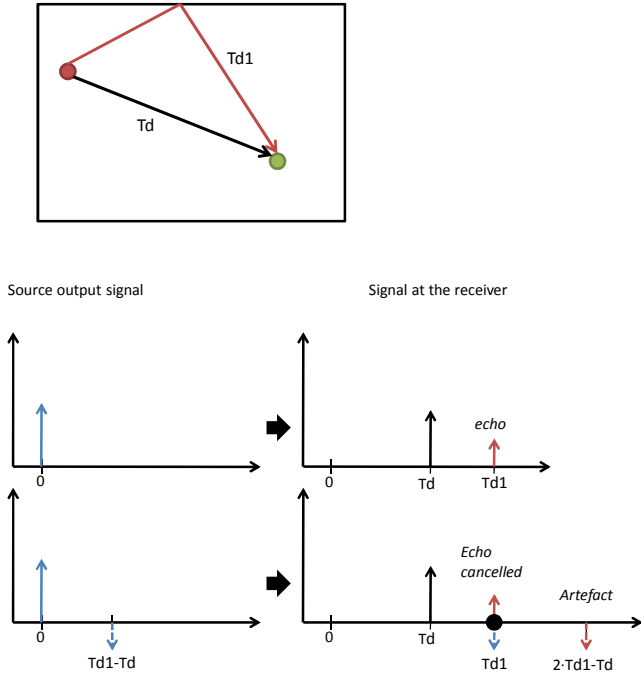


Figure 5 – simplified diagram of room reflections correction

Lets consider a room with source and a receiver, and only two propagations paths, as depicted in Figure 5. T_d is the time delay associated with direct arrival of sound, T_{d1} is time delay of reflected arrival, $T_{d1} > T_d$. To simplify the discussion we'll model sound pulses as ideal delta function, and only consider single reflection. Source emits an ideal pulse at $t=0$. At T_d , the direct signal will arrive at the receiver. At T_{d1} , signal reflected off the wall will arrive at the receiver – the echo. We can try to cancel it by pre-distorting the original signal. Direct path correction signal will arrive at the same time and opposite phase as the original reflected one and cancel out.

This produces a problem since the cancellation signal will travel along the path T_{d1} as well as T_d and occur later as un-cancelled artefact.

This diagram and discussion are very simplified. In reality there is a very large number of reflections even in a simple environment, and input signal is never just a pulse but music. Nevertheless it shows that dealing with acoustic echoes is not trivial even for the simplest case.

The diagram of channel equalisation is shown in Figure 6. Considering D-to-A and A-to-D converters ideal for now, the aim of the system is to make $d(t) = x(n - D)$, in other words make signal at listener's position an exact and delayed reproduction of input signal $x(n)$ in analogue domain.

I assume that we can exactly measure channel c . Then, having the knowledge of c , we want to adjust filter w so that $x \otimes w \otimes c \otimes = x = d$ In simple terms, the problem is finding w such that $w \otimes c = 1$.

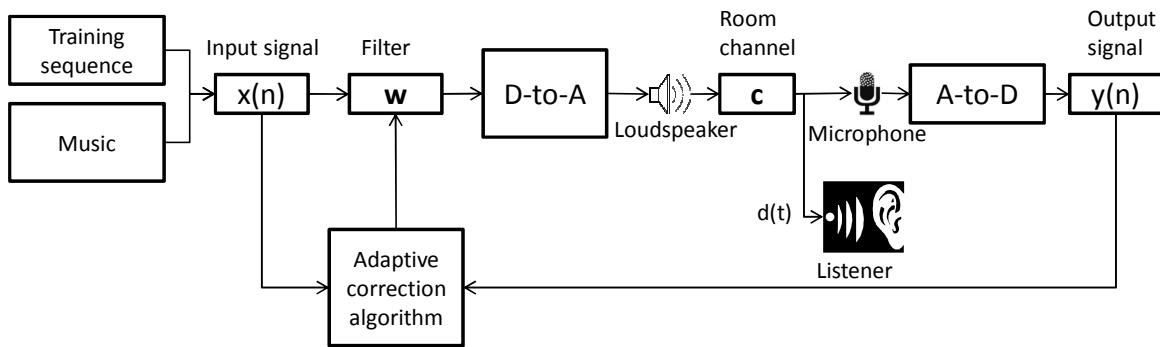


Figure 6

The parameters of the system that I have to take into consideration are summarised in Table 1. These are only typical parameters and can vary in real system, however these give me a good starting point if I wish to simulate the performance of the system.

Table 1

System block	Parameters	Typical value
Filter	- FIR or IIR type - Number of taps	Fixed point FIR 1000's of taps
D-to-A	- THD - SNR	0.01% 110dB at 1V output
Loudspeaker	- One, two or there way - Frequency response (direction dependant) - Sensitivity	Two way (tweeter and a bass speaker) Quite directional for high frequencies 85dB @ 2.83V/1m
Room channel	- Impulse response - Length in time	Mixed-phase transfer function 0.1s-2s to -60dB fall-off
Microphone	- Frequency response (direction dependant) - Sensitivity	Omni-directional -42 db (7.9 mV) re 1V at 1 Pa
A-to-D	- Resolution(bits) - SNR - Sample rate	16 bits 85dB 44.1KHz

I attempted to perform impulse cancellation by using of LMS algorithm and by directly inverting the impulse response using IIR filtering.

I have made use of Matlab script [7] to compare the two ways of cancelling impulse response. First, I will analyse the maths behind the problem.

Maths

The output of a linear system is just time-domain convolution of input signal with impulse response: $y = x \otimes h$. We'll use Z-domain because the sound is stored and processed in digital domain. The output is found by multiplication $Y(z) = X(z)H(z)$. If we want to obtain the original input signal we can multiply the channel transfer function by its inverse:

$$Y(z) = X(z)H(z)H^{-1}(z) = X(z)$$

The Z-transform of a sampled sequence can be found as $H(z) = \sum_{n=0}^{\infty} h(n)z^{-n}$, where z is complex variable.

We can factorise the numerator and denominator of $H(z)$ to find its roots:

$$H(z) = \frac{(z-z_0)(z-z_1)\dots}{(z-p_0)(z-p_1)\dots}, \text{ where } z_0, z_1\dots \text{ are zeros of the system and } p_0, p_1\dots \text{ are poles of the system.}$$

The requirement for the system to be stable in Z-domain is that all the poles of the system must lie within unity circle on the complex Z-plane. Zeros can be outside the unit circle.

Now if we try to obtain $H^{-1}(z)$, poles and zeros swap places:

$$H^{-1}(z) = \frac{(z-p_0)(z-p_1)\dots}{(z-z_0)(z-z_1)\dots}$$

This would cause the inverse of any system that had zeros outside unit circle to have poles outside unit circle, which means the system would be unstable and oscillate.

$h(t)$ of a room is shown in Figure 4. The $H(z)$ must be stable as the value of $h(t)$ decays to 0 with time. However zero-pole plot of $H(z)$ examination in Figure 7 reveals that the transfer function contains zeros outside unit circle:

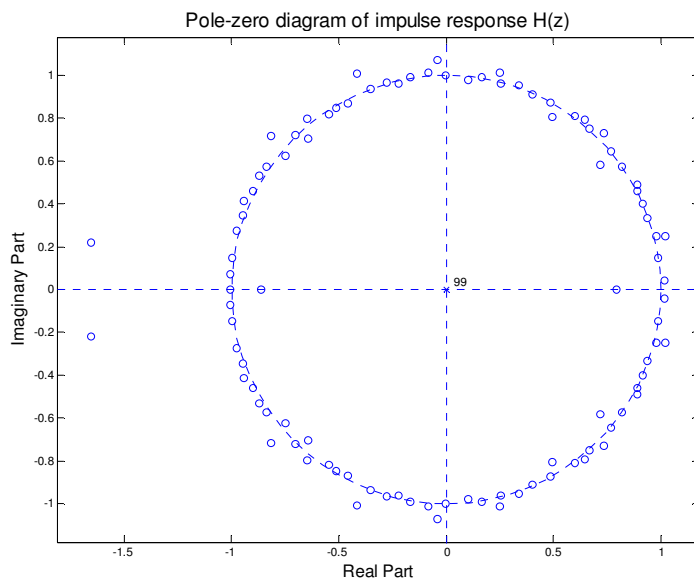


Figure 7 – pole-zero plot of first 100 elements of recorded impulse response

That means that $H^{-1}(z)$ will contain poles outside unit circle and be unstable. Therefore multiplication $H(z)H^{-1}(z)$ will not give us the desired result, as the output will become very large (undetermined).

Matlab function `filter` implements digital filter which transfer function is

$$Y(z) = \frac{b(1) + b(2)z^{-1} + \dots + b(nb+1)z^{-nb}}{1 + a(2)z^{-1} + \dots + a(na+1)z^{-na}} X(z)$$

If we set all the a coefficients to 0, let $b[n]=h[n]$ and let $X(z)$ be delta function i.e. $[1,0,0,\dots]$ then the output of the filter will be just $h(n)$. In this case filter command effectively performs multiplication by $H(z)$ (numerator is just Z transform of vector $\mathbf{b}$). We can use this command to find $H^{-1}(z)$. To do this we set $b(1)$ to 1, and let $a(n)=h(n)$. Now the filter command will perform multiplication by H^{-1} in Z-domain.

Figure 8a shows a simple channel response $h=[1 \ 0 \ 0 \ 0.5 \ 0 \ 0.25]$. The corresponding inverse equaliser filter (IF) is obtained by inverting $H(z)$: `filter(1, h, [1 0 0...])`. It is shown in Figure 8b.

By visually inspecting the result we see that at the beginning the equalizer filter is just negative of the channel: $IF[n]=-h[n]$, $n=2..6$. FIR filter is obtained just by truncating IIR filter coefficients to a certain length, 32 in this case (Figure 8c).

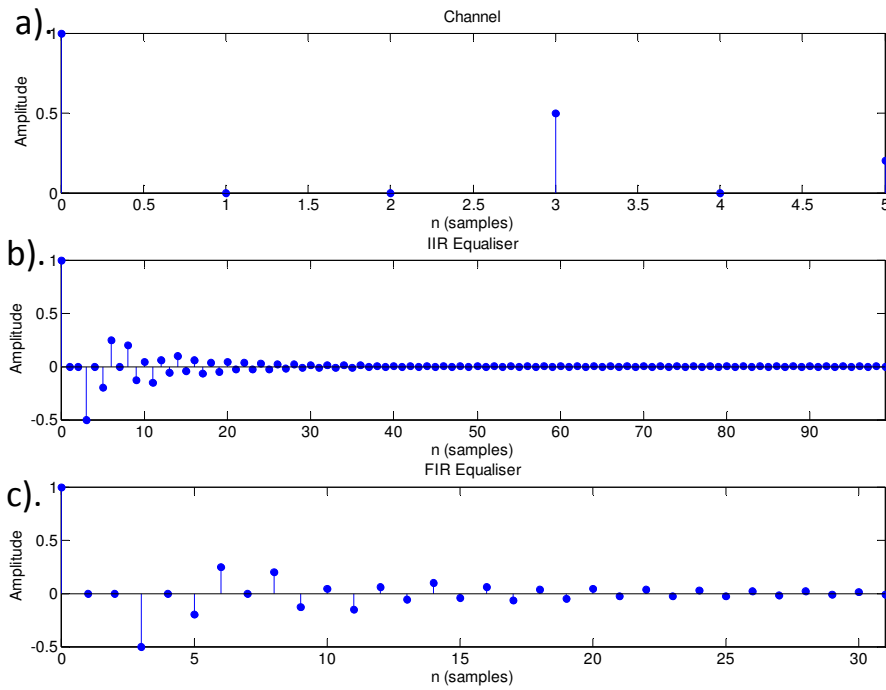


Figure 8

Figure 9a shows a white noise signal processed by an inverse FIR filter and LMS adaptive filter. Filter lengths are 32. The signal after the channel is not shown.

We can see that FIR filter equalises the signal very well, with RMS (Root Mean Squared) error of 1.89%.

The LMS algorithm is discussed in detail in [8], [9]. It is an adaptive algorithm that adjusts the coefficients of a digital filter based on the difference between actual and desired output.

LMS performs rather poorly in this situation, for the step size $\mu=0.01$. RMS error is 49%. Changing step size did not improve the performance.

Variance of filter coefficients $\mathbf{w}$ shown in Figure 9c is just sum of the coefficients, a simple way to visualise how much filter is changing in time.

It shows that filter does not converge.

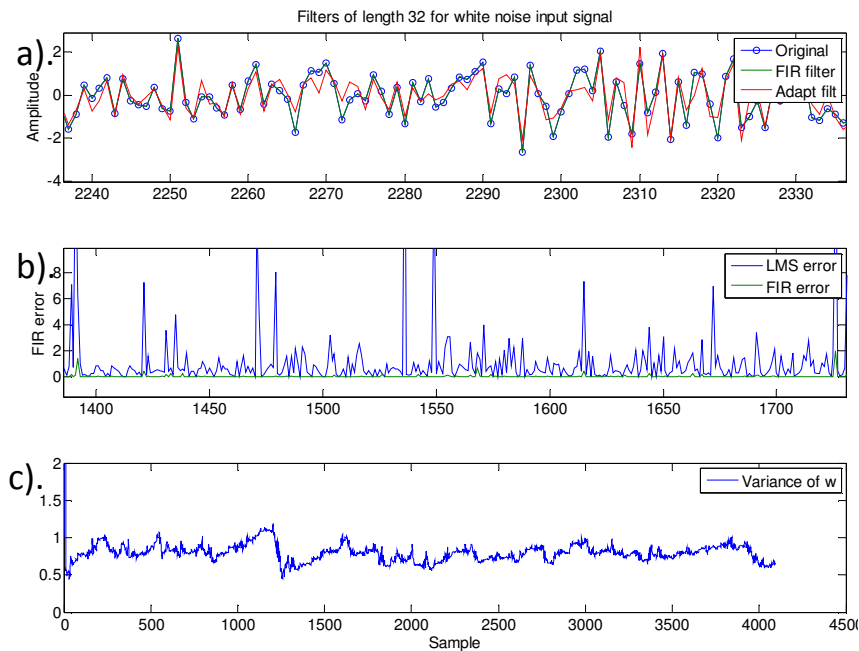


Figure 9

Figure 10 shows the first 120 non-zero elements of the recorded impulse response. It is only a very short sample of the entire impulse response, but length has to be kept short to avoid excessive processing times.

We can see that the inverse filter is unstable and starts to oscillate with ever-increasing amplitude. Therefore it can not be used for equalisation.

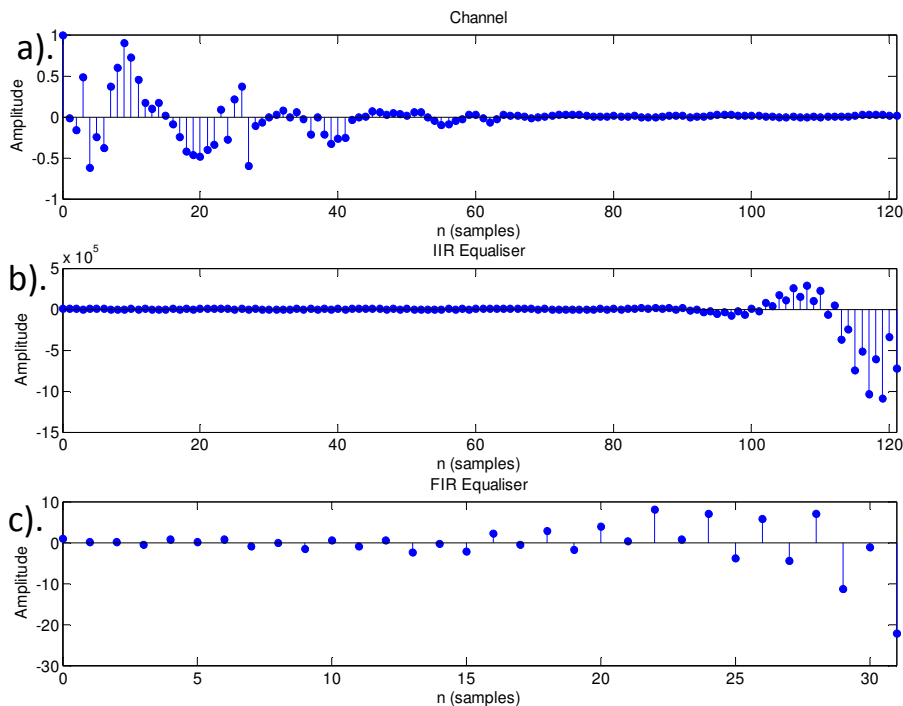


Figure 10 – recorded impulse response and inverse filters

I attempted to equalise a sample of real music using LMS algorithm. The piece of music was approx. 49000 samples at 44.1KHz or 11 seconds long. I have used impulse response shown in Figure 10a. Length of equalisation filter was set to 1000 coefficients, and step size was varied through 3 values in this order: 1, 0.5, 0.2, gradually refining the coefficients.

Figure 11a shows the original signal and signal equalised by LMS. Figure 11b shows the relative error of LMS iterations. The final RMS value of the error is 0.015, or 1.5%. This is quite good, however RMS value does not tell the whole picture about the acoustical performance of the filter. Close inspection shows that equalised output follows slow changing signal well, but skips the sharp peaks. Equalisation filter therefore acts as low-pass filter. Inspection of FFT spectrum of original and restored signal confirms this observation.

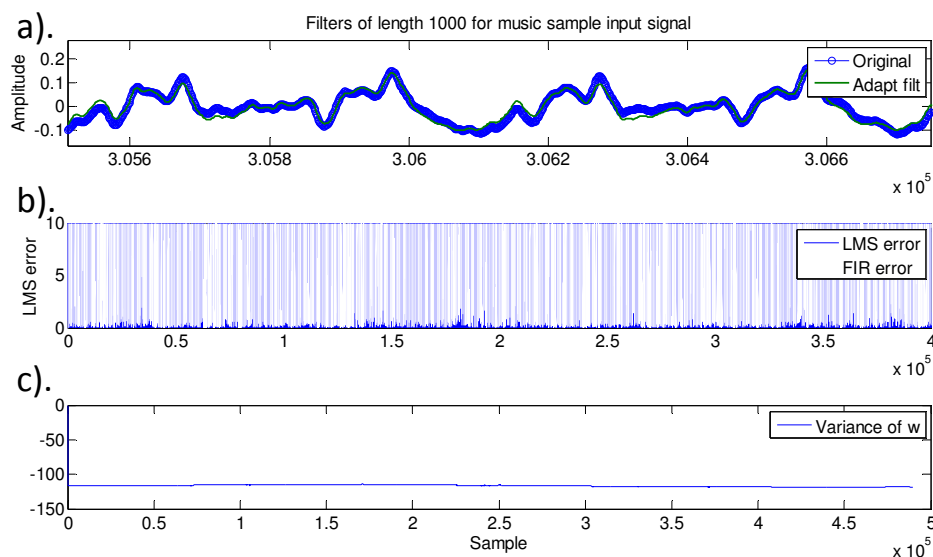


Figure 11 – equalisation of the impulse response using LMS

To achieve good results even for relatively short impulse responses large number of filter coefficient are needed. For channel length of 6(samples), FIR filter of length 32 performed well. For a impulse length of 120 samples, filter length was 1000 coefficients to achieve acceptable performance. This makes filter length ≈ 6 times the channel length.

The total length of simulated impulse response is approximately 0.2s, so for a sample rate of 44100 Hz the correction filter would have between 40000 and 50000 taps. Clearly that is far too long to be practical, especially if system cost to be kept reasonable.

Listening tests give impression that equalisation does restore the music towards the original. However low-pass effect of the filter is quite audible.

Further work

Further work on equalisation will encompass two main directions: perfecting LMS algorithm, and researching a way to convert mixed-phase transfer function of the room in minimal phase system. The final method may well be combination of the two, or a different method altogether.

So far I have assumed mono sound source. In practice, majority of hi-fi systems have 2 loudspeakers. I will research into how both of the loudspeakers can be used to enhance the equalisation.

To implement the system in real-time, reduction in number of filter taps must be achieved in some way.

Conclusion

Whilst researching and developing an acoustic room simulation model I fully realised the complexity of the subject. Even a moderately accurate model will require significantly more time and effort than the 3rd year project allows. Plus, the accuracy of the model does not play crucial role in the project – room equalisation. I can just physically measure the accurate impulse response and work with that.

Research and work on a room equalisation system challenged my understanding of digital filters. The transfer function of the room is almost certainly a mixed phase system, which means that a cancellation filter designed by simple inversion of the room transfer function will be unstable. That was confirmed by my simulations.

A Least Mean Squares (LMS) adaptive algorithm was used to tackle the problem. It produced somewhat encouraging results, but more work is needed. Any equalisation system is a compromise, so further measurements and subjective tests are needed to determine if equalisation has positive overall impact on music perception.

Revised schedule

For the presentation and the final report I aim to:

- Develop a way to measure impulse response of the room using swept sine technique.
- Refine equalisation algorithm and decide if it provides objective and subjective enhancement of the sound.
- Develop a way to automatically compute active cross-over digital filters for a loudspeaker, so achieve maximally flat frequency response.

Bibliography

[1] Survey of Methods for Modeling Sound Propagation in Interactive Virtual Environment Systems. [Online]. <http://www-sop.inria.fr/rees/Nicolas.Tsingos/publis/presence03.pdf>

[2] Heinrich Kuttruff, *Acoustics an Introduction.*: Taylor and Francis, 2007.

- [3] Andrew S. Glassner, *An Introduction to ray tracing*, 3rd ed.: Academic Press, 1990.
- [4] Ingrid Carlbom, Gary Elko, Thomas Funkhouser and Robert Kubli Nicolas Tsingos, "Validation of Acoustical Simulations in the "Bell Labs Box",".
- [5] T. Funkhouser, A. Ngan, and I. Carlbom N. Tsingos, "Modeling acoustics in virtual environments using the uniform theory of diffraction," August 2001.
- [6] Marco Jeub. RWTH Aachen University, Institute of Communication Systems and Data Processing (IND).
<http://www.ind.rwth-aachen.de/AIR>.
- [7] Benn Thomsen. (November 2010) Digital Channel Equalisation Example.
- [8] B. Farhang Boroujeny, *Adaptive filters theory and applications*.: Wiley, 1999.
- [9] Simon Haykin, *Communication Systems*, 3rd ed., 1994.
- [10] Markus Kallinger and Alfred Mertins, "ROOM IMPULSE RESPONSE SHORTENING BY CHANNEL SHORTENING CONCEPTS,".